

codigo final chatgpt

```
import statistics

# Constantes do sistema
TENSAO_NOMINAL = 220
TENSAO_MINIMA = 0
TENSAO_MAXIMA = 500

def validar_medicao(valor):
    """
    Verifica se a medição é numérica e está na faixa válida.
    """
    try:
        valor = float(valor)
        return TENSAO_MINIMA <= valor <= TENSAO_MAXIMA
    except (ValueError, TypeError):
        return False

def calcular_desvio_percentual(tensao):
    """
    Calcula o desvio percentual em relação à tensão nominal.
    """
    return abs(tensao - TENSAO_NOMINAL) / TENSAO_NOMINAL * 100

def classificar_medicao(tensao):
    """
    Classifica a medição em NORMAL, ALERTA ou CRÍTICO.
    """
    desvio = calcular_desvio_percentual(tensao)

    if desvio > 20:
        return "CRITICO"
    elif desvio > 10:
        return "ALERTA"
    else:
        return "NORMAL"

def filtrar_medicoes(medicoes_brutas):
    """
    Separa medições válidas e contabiliza descartes.
    """
    validas = []
    descartadas = 0
```

```

for valor in medicoes_brutas:
    if validar_medicao(valor):
        validas.append(float(valor))
    else:
        descartadas += 1

```

```

return validas, descartadas

```

```

def calcular_estatisticas(medicoes):
    """
    Calcula média, mínimo, máximo e desvio padrão.
    """
    return {
        "media": statistics.mean(medicoes),
        "minimo": min(medicoes),
        "maximo": max(medicoes),
        "desvio_padrao": (
            statistics.stdev(medicoes)
            if len(medicoes) > 1 else 0
        )
    }

```

```

def classificar_medicoes(medicoes):
    """
    Retorna a classificação de cada medição válida.
    """
    return [classificar_medicao(valor) for valor in medicoes]

```

```

def detectar_eventos_criticos(classificacoes):
    """
    Conta eventos críticos e encontra a maior sequência consecutiva.
    """
    quantidade_criticos = 0
    sequencia_atual = 0
    maior_sequencia = 0

    for estado in classificacoes:

        if estado == "CRITICO":
            quantidade_criticos += 1
            sequencia_atual += 1

        if sequencia_atual > maior_sequencia:
            maior_sequencia = sequencia_atual

```

```

        else:
            sequencia_atual = 0

    return quantidade_criticos, maior_sequencia

def gerar_relatorio(
    estatisticas,
    quantidade_validas,
    quantidade_descartadas,
    classificacoes,
    eventos_criticos,
    maior_sequencia):
    """
    Exibe o relatório final.
    """

    print("\n===== RELATÓRIO =====")

    print(f"\nAmostras válidas: {quantidade_validas}")
    print(f"Amostras descartadas: {quantidade_descartadas}")

    print("\n--- Estatísticas ---")
    print(f"Média: {estatisticas['media']:.2f} V")
    print(f"Mínimo: {estatisticas['minimo']:.2f} V")
    print(f"Máximo: {estatisticas['maximo']:.2f} V")
    print(f"Desvio padrão: {estatisticas['desvio_padrao']:.2f} V")

    print("\n--- Classificação ---")
    print(f"NORMAL : {classificacoes.count('NORMAL')}")
    print(f"ALERTA : {classificacoes.count('ALERTA')}")
    print(f"CRITICO: {classificacoes.count('CRITICO')}")

    print("\n--- Eventos Críticos ---")
    print(f"Quantidade de eventos: {eventos_criticos}")
    print(f"Maior sequência crítica: {maior_sequencia}")

    print("\n=====")

def main():
    """
    Função principal do programa.
    """

    medicoes_brutas = [
        220,

```

```
230,  
180,  
170,  
250,  
280,  
290,  
"erro",  
-10,  
510,  
225,  
215  
]
```

```
medicoes_validas, descartadas = filtrar_medicoes(medicoes_brutas)
```

```
if not medicoes_validas:  
    print("Nenhuma medição válida encontrada.")  
    return
```

```
estatisticas = calcular_estatisticas(medicoes_validas)
```

```
classificacoes = classificar_medicoes(medicoes_validas)
```

```
eventos_criticos, maior_sequencia = detectar_eventos_criticos(  
    classificacoes  
)
```

```
gerar_relatorio(  
    estatisticas,  
    len(medicoes_validas),  
    descartadas,  
    classificacoes,  
    eventos_criticos,  
    maior_sequencia  
)
```

```
if __name__ == "__main__":  
    main()
```